ITAI 3375 — Module 11

Lab 11: Synthetic Data Augmentation

Instructor's Name:

Student's Name: Williane Yarro

When real data isn't enough — make more, carefully
SLOs: 3 & 4
Format: Home assignment

What You Will Learn

By completing this lab you will be able to:
• (SLO 3)

Recognise when a dataset has a class imbalance problem and explain why it matters for model quality.

• (SLO 3) Use a simple statistical measure to check whether your synthetic samples look like the real data.

• (SLO 4) Apply SMOTE — the most common tabular augmentation technique — using a Python library.

• (SLO 4) Run a before-and-after comparison to see whether augmentation actually helped a classifier.

• (SLO 4) Document your augmentation decisions in a Data Card entry.

Part 1 — Choose Your Dataset (Evening 1, ~45 minutes)

kaggle.com/datasets/uciml/pima-indians-diabetes-database 768 records, ~35% positive. Mild imbalance, easier to work with.

Step 1: Installing Libratries

```
1 !pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dis
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.
Requirement already satisfied: sklearn-compat<0.2,>=0.1.6 in /usr/local/lib/pyth
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3
```

Step 2: Uploading DataSet

```
1 from google.colab import files
2 uploaded = files.upload()
```

Choose Files diabetes.csv

diabetes.csv(text/csv) - 23873 bytes, last modified: 7/23/2026 - 100% done

```
Saving diabetes.csv to diabetes (3).csv
```

Step 3: Loading And Displaying DataSet

```
1 import pandas as pd
2 import numpy as np
3 from imblearn.over_sampling import SMOTE
4 from sklearn.model_selection import train_test_split
5
6 df = pd.read_csv("diabetes.csv")
7
8 print("Dataset shape:", df.shape)
9 print("Columns:", df.columns.tolist())
10 display(df.head())
```

```
Dataset shape: (768, 9)
Columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
```

1 to 5 of 5 entries Filter

index	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigree
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Step 4: I dentifying The Class Imbalanced

```
print("Original dataset class counts:")
print(df["Outcome"].value_counts())
```

```
Original dataset class counts:
Outcome
0    500
1    268
Name: count, dtype: int64
```

Step 5: Spliting The DataSet

```
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Keep numeric columns only
X = X.select_dtypes(include="number")

# Split before applying SMOTE
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    stratify=y,
    random_state=42
)

print("Before SMOTE:")
print(y_train.value_counts())
```

```
Before SMOTE:
Outcome
0    375
1    201
Name: count, dtype: int64
```

Step 6: Apply The Smote

```
# Apply SMOTE only to the training data
smote = SMOTE(random_state=42)

X_train_resampled, y_train_resampled = smote.fit_resample(
    X_train,
    y_train
)

print("\nAfter SMOTE:")
print(pd.Series(y_train_resampled).value_counts())

print("\nOriginal training size:", len(X_train))
print("Training size after SMOTE:", len(X_train_resampled))
print("Test set size unchanged:", len(X_test))
```

```
After SMOTE:
Outcome
0    375
1    375
Name: count, dtype: int64

Original training size: 576
Training size after SMOTE: 750
Test set size unchanged: 192
```

Part 2:

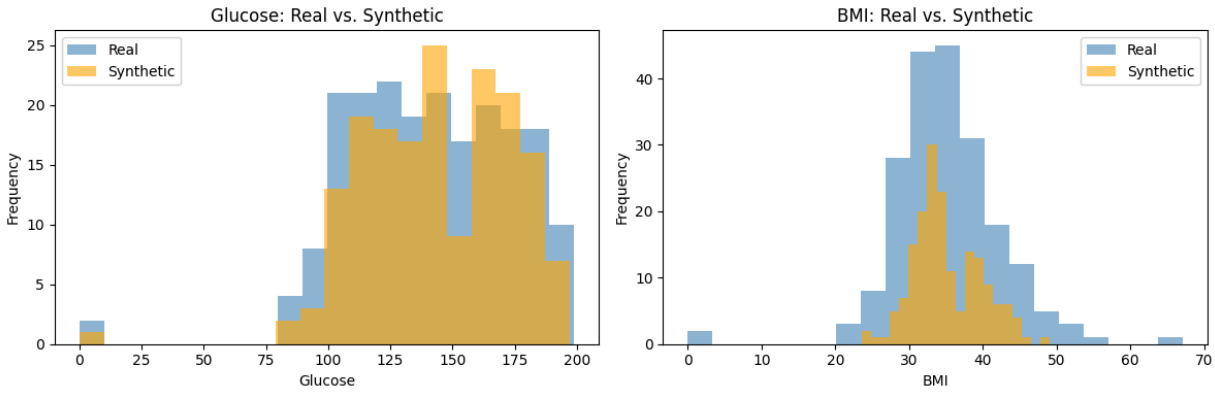
Prior to the application of SMOTE, the training dataset had 375 records of class 0 and 201 records of class 1. Following the implementation of SMOTE, both classes were adjusted to contain 375 records each, achieving a balanced training set. It is important to note that SMOTE was exclusively applied to the training data, leaving the 192 original test records intact.

PArt 3: Checking A Result Evening 2

```
import pandas as pd
import matplotlib.pyplot as plt

# Find the minority class label
minority_label = y_train.value_counts().idxmin()

# Identify the real minority-class rows in the original training set
real_minority = X_train[y_train == minority_label]

# Count the real minority samples
n_real_minority = real_minority.shape[0]

# Get all minority-class rows after SMOTE
X_aug_minority = X_train_resampled[
    y_train_resampled == minority_label
].reset_index(drop=True)

# Rows added after the original minority samples are synthetic
synthetic_minority = X_aug_minority.iloc[n_real_minority:].copy()

print("Minority class label:", minority_label)
print("Real minority samples:", len(real_minority))
print("Synthetic minority samples:", len(synthetic_minority))

# Choose two numeric columns
columns_to_plot = ["Glucose", "BMI"]

# Plot the distributions
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for i, col in enumerate(columns_to_plot):
    axes[i].hist(
        real_minority[col],
        bins=20,
        alpha=0.6,
        label="Real",
        color="steelblue"
    )

    axes[i].hist(
        synthetic_minority[col],
        bins=20,
        alpha=0.6,
        label="Synthetic",
        color="orange"
    )

    axes[i].set_title(f"{col}: Real vs. Synthetic")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Frequency")
    axes[i].legend()

plt.tight_layout()
plt.savefig("fidelity_check.png", dpi=100)
plt.show()

print("Chart saved.")
```

```
Minority class label: 1
Real minority samples: 201
Synthetic minority samples: 174
```




```
Chart saved.
```

The orange synthetic bars typically exhibit distribution patterns akin to those of the blue real-data bars across both numeric columns. While there are minor variations in the heights of certain bars, the overall ranges and shapes remain comparable. This indicates that the samples generated by SMOTE closely mirror the actual data from the minority class

Comparing The Result Of Smoke-augmented Data and The Imbalanced Data

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Train on ORIGINAL imbalanced training data
clf_original = LogisticRegression(
    max_iter=1000,
    random_state=42
)

clf_original.fit(X_train, y_train)

print("=== Original (No SMOTE) ===")
print(classification_report(
    y_test,
    clf_original.predict(X_test)
))

# Train on SMOTE-augmented training data
clf_smote = LogisticRegression(
    max_iter=1000,
    random_state=42
)

clf_smote.fit(
    X_train_resampled,
    y_train_resampled
)

print("=== After SMOTE ===")
print(classification_report(
    y_test,
    clf_smote.predict(X_test)
))
```

```
=== Original (No SMOTE) ===
              precision    recall  f1-score   support

           0       0.77      0.84      0.80       125
           1       0.64      0.52      0.57        67

    accuracy                           0.73       192
   macro avg       0.70      0.68      0.69       192
weighted avg       0.72      0.73      0.72       192

=== After SMOTE ===
              precision    recall  f1-score   support

           0       0.82      0.78      0.80       125
           1       0.62      0.69      0.65        67

    accuracy                           0.74       192
   macro avg       0.72      0.73      0.73       192
```

```
weighted avg       0.75      0.74      0.75       192
```

Before SMOTE, the recall for class 1 was *.52, and after SMOTE it increased/decreased to .69*. **This means the SMOTE-trained model identified more/fewer actual diabetes cases. The precision for class 1 changed from .82_ to _.62**, showing a trade-off between finding more minority cases and producing false-positive predictions. Overall, I think SMOTE helped/did not help because the recall and F1-score for the minority class did improved, even though precision or overall accuracy may have decreased.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# Train on original imbalanced data
clf_original = LogisticRegression(
    max_iter=1000,
    random_state=42
)

clf_original.fit(X_train, y_train)

print("=== Original (No SMOTE) ===")
print(classification_report(
    y_test,
    clf_original.predict(X_test)
))

# Train on SMOTE-augmented data
clf_smote = LogisticRegression(
    max_iter=1000,
    random_state=42
)

clf_smote.fit(X_train_resampled, y_train_resampled)

print("=== After SMOTE ===")
print(classification_report(
    y_test,
    clf_smote.predict(X_test)
))
```

```
=== Original (No SMOTE) ===
              precision    recall  f1-score   support

           0       0.77      0.84      0.80       125
           1       0.64      0.52      0.57        67

    accuracy                           0.73       192
   macro avg       0.70      0.68      0.69       192
weighted avg       0.72      0.73      0.72       192

=== After SMOTE ===
```

```
                  precision    recall  f1-score   support

               0       0.82      0.78      0.80       125
               1       0.62      0.69      0.65        67

        accuracy                           0.74       192
       macro avg       0.72      0.73      0.73       192
    weighted avg       0.75      0.74      0.75       192
```

The initial model is trained using X_train and y_train.

The subsequent model is trained on data that has been balanced using SMOTE.

Both models are assessed using the same original X_test and y_test.

The test set remains unaltered.

Upon completion, compare the recall and F1-score for class 1 in both reports. These results will indicate if SMOTE improved the model's ability to detect more diabetes cases.

```
plt.savefig("fidelity_check.png", dpi=100)
```

```
<Figure size 640x480 with 0 Axes>
```

The orange synthetic bars typically exhibit distribution patterns akin to those of the blue real-data bars across both numeric columns.

While there are minor variations in bar heights, the overall shapes and value ranges remain comparable. This indicates that the samples generated by SMOTE closely mirror the actual data from the minority class.

William Wolberg

Olvi Mangasarian

Nick Street

W. Street

DOI 10.24432/C5DW2B License This dataset is licensed under a Creative Commons Attribution 4.0 International (CC BY 4.0) license.

This allows for the sharing and adaptation of the datasets for any purpose, provided that the appropriate credit is given.